Web Framework Development

Year 3 Computing

TU764 / TU860 / TU757

In-class Test

Semester 2 2025/26

Date of examination	Monday 23rd March 2025
Time of assessment	14:00 – 16:00 The test duration is 90 minutes
Location of assessment	Room BD025
Internal examiner(s)	Dr. Anh Duong Trinh

Instructions to candidates:

1	This is an **individual** test – all work should be your own. During the test you should NOT communicate with anyone apart from the lecturer (except medical / emergency circumstances).
2	NO communication with others is permitted (e.g. to NOT use phones, email, Google, etc.)
3	You may use your own laptop for this test. NOTE: You are responsible for the working conditions of your own device.
4	This is an open-book test. You may use lecture notes and lab materials to perform this test. You are NOT allowed to use online materials and generative AI tools during the test.
5	It is recommended that you answer the questions in the sequence they appear (Part A & B), since some questions rely on your work created in earlier questions.
6	The test is marked out of 100. Answer ALL questions.

PART A – Object-Oriented Programming (30 marks)

Create a python file named **PartA.py**. This file will contain all the codes you create for Python OOP programming.

Exercise:
Define and implement the classes with initial attributes required to represent a music playlist:

Class	Initial Attributes
Artist	name, DoB, country, list of albums, list of songs
Song	song title, artist's name, year of release
Album	album title, artist's name, year of release, list of songs
Playlist	playlist title, list of songs

Basic requirements:

- A Song and an Album should be associated with an Artist by the artist's name.
- By default, the all the list attributes should be initialised as empty.
- The Artist, Song and Album classes should implement a method ***display_info()*** to display all the attributes and their values.
- The Album class should implement a method ***add_song()*** that takes a title and a release year and creates a new song which is then added to its list of songs.
- The Artist class should implement two methods ***add_album()***, which takes an album as a parameter, and ***add_song()***, which takes a song as a parameter.
- The Playlist class should implement a method ***add_song()*** that adds a song to its list of songs.

Advanced features:
Implement the following methods in the Playlist class:

- ***print_all_song()***: print all the songs in the playlist.
- ***sort_playlist(order='ASC')***: sort all the songs in the playlist by the name either in ascending or descending order (order = 'ASC' or order = 'DES').
- ***shuffle_playlist()***: shuffle all the songs in the playlist

Further actions:
You can think of your own examples or use the provided example (*TaylorSwift.pdf*) to perform these actions.

- Create an artist
- Create an album
- Create a few songs for the artist you have created
- Use method ***add_song()*** from the Album class to add two songs
- Use methods ***add_album()*** and ***add_song()*** to update the information of the

artist
- Create a playlist
- Add all songs from the album to the playlist

<u>Finally:</u>
Demonstrate how your implemented classes and methods work.

PART B – Unit Testing (30 marks)

Create a python file named **PartB.py**. This file will contain all the codes you create for Python unit tests based on Part A exercises.

Exercise:

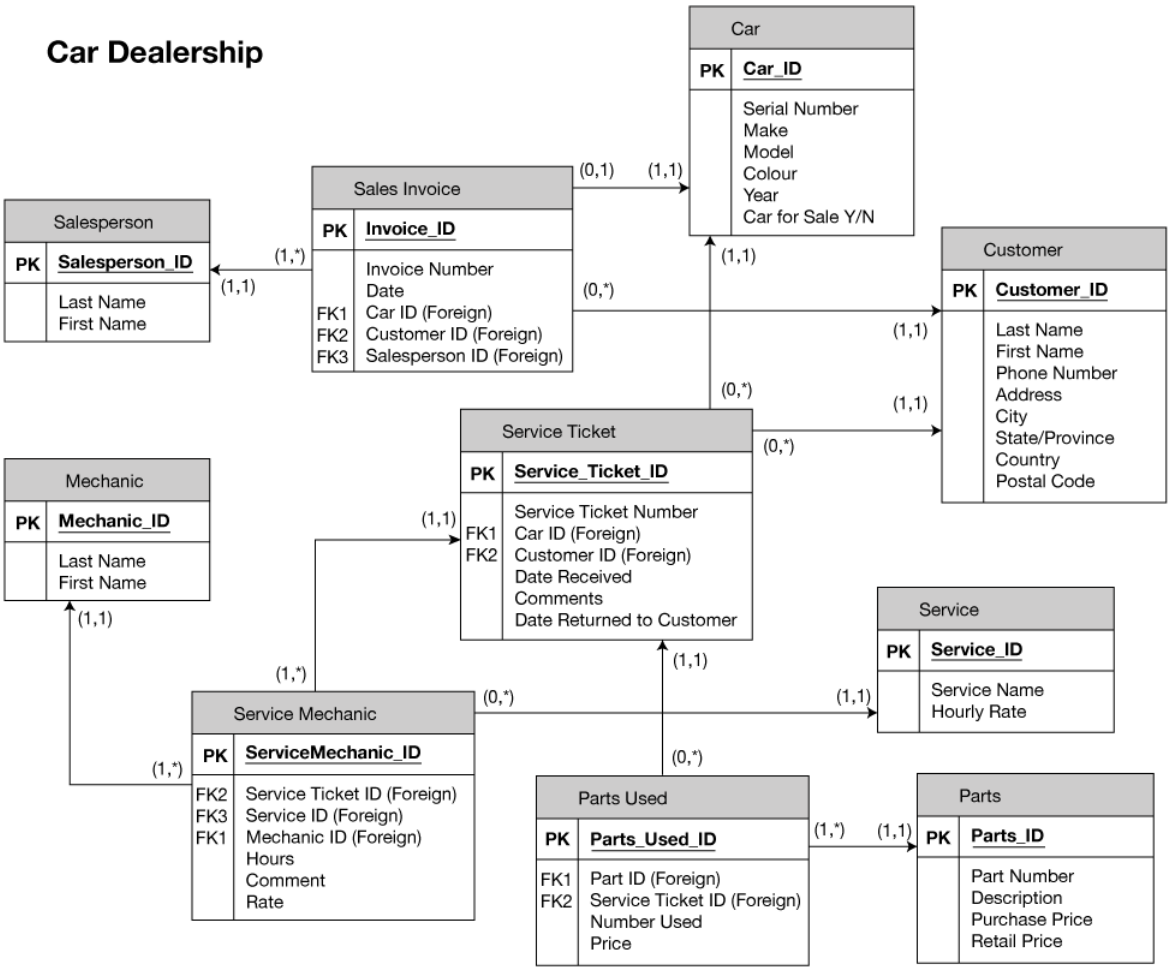
- Write unit tests to test if an object is an instance of a class for all classes (4 tests).
- Write unit tests to test if an object is NOT an instance of a class for all classes (4 tests).
- Write 2 unit tests to test if 2 objects are identical, where 1 unit test is for identical objects and 1 unit test is for unidentical but similar objects (2 tests).
- Write unit tests to test the work of all methods ***add_song()*** and ***add_album()*** (4 tests).
- Write unit tests to test the sorting methods of Playlist class, namely ***sort_playlist()*** and ***shuffle_playlist()*** (3 test)

Finally:

Run and pass all the tests.

PART C – Django Model Layer (30 marks)

You are provided with an ERD for car dealership database.



Exercise:

- In your Django project please create **models.py**
- Create models based on the ERD above.

Please try your best to create as many models as possible. Each model correctly created will give you marks.

PART D – Github Commit / Push (10 marks)

Your Github submission must contain 3 files:

- PartA.py
- PartB.py
- models.py

Github commit / push with timestamps after the test time will **NOT** be accepted.

Github repository should be **public**.

The submission on Brightspace is a text file containing the URL to your Github repository.